

Esercitazione Laravel — WorkforceApp

Introduzione

Questo documento descrive l'esercitazione pratica del corso di **Tecnologie Web**.

L'esercitazione si svolge su **WorkforceApp**, un'applicazione Laravel per la gestione di dipendenti, dipartimenti e progetti aziendali. Il codice sorgente è **parzialmente incompleto**: alcune sezioni sono state rimosse intenzionalmente e devono essere completate dallo studente.

Le lacune sono segnalate da commenti nel formato `// TASK N` — all'interno dei file sorgente. Ogni task indica esattamente quale file modificare e quale comportamento implementare.

Obiettivo: completare tutte le sezioni mancanti in modo che l'applicazione funzioni correttamente nella sua interezza.

Prima di iniziare

1. Prepara un database vuoto

2. Installa le dipendenze e prepara l'ambiente

```
npm install
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
```

3. Esegui il progetto

Esegui i comandi in 2 terminali separati

```
npm run dev  
php artisan serve
```

Sezione 1 — Modello e Migrazione

File da modificare:

- `app/Models/Project.php`
- `database/migrations/*_create_projects_table.php`

TASK 1 — Relazione Project → Department

File: `app/Models/Project.php`

Un progetto appartiene a un dipartimento. Completa il metodo `department()` nella classe `Project` definendo la relazione Eloquent corretta.

Relazione da usare: `belongsTo`

Modello correlato: `Department`

TASK 2 — Relazione Project ↔ Employee (many-to-many)

File: `app/Models/Employee.php`

File: `app/Models/Project.php`

Un progetto può coinvolgere più dipendenti e un dipendente può lavorare su più progetti. La tabella pivot `employee_project` contiene anche la colonna `hours`.

Completa il metodo `employees()` nella classe `Project` :

Completa il metodo `projects()` nella classe `Employee` :

- usa la relazione `belongsToMany` verso `Employee` e `Project`
- aggiungi `withPivot('hours')` per rendere accessibile il campo ore
- aggiungi `withTimestamps()` per gestire i timestamp della tabella pivot

TASK 3a — Cancellazione a cascata nella migrazione

File: `database/migrations/*_create_projects_table.php`

Quando un dipartimento viene eliminato, tutti i progetti ad esso associati devono essere eliminati automaticamente dal database.

Completa la definizione della foreign key `department_id` aggiungendo il metodo che attiva la cancellazione a cascata.

Suggerimento: il metodo si chiama `cascadeOnDelete()` e va concatenato alla definizione della foreign key.

TASK 3b — Aggiunta del Seeder per il Project

Nel `DatabaseSeeder` (`database/seeder/DatabaseSeeder.php`) aggiungi il `ProjectSeeder` all'elenco dei Seeder eseguiti.

⚠ Dopo aver modificato la migrazione e aggiunto il seeder, riesegui
`php artisan migrate:fresh --seed` per applicare le modifiche.

Sezione 2 — Controller CRUD

File da modificare: `app/Http/Controllers/ProjectController.php`

TASK 4 — Validazione in `store()`

File: `app/Http/Controllers/ProjectController.php` → metodo `store()`

Completa l'array passato a `$request->validate([...])` con le seguenti regole:

Campo	Regole
<code>name</code>	obbligatorio, stringa, min 3 caratteri, max 255 caratteri
<code>site_name</code>	facoltativo, stringa, min 2 caratteri, max 255 caratteri
<code>department_id</code>	obbligatorio, deve esistere nella tabella <code>departments</code>

TASK 5 — Salvataggio del progetto in `store()`

File: `app/Http/Controllers/ProjectController.php` → metodo `store()`

Dopo la validazione, crea un nuovo record `Project` nel database usando i dati dell'input. I dati sono già raccolti nella variabile `$input`.

Suggerimento: usa il metodo statico `Project::create($input)`.

TASK 6 — Validazione e aggiornamento in `update()`

File: `app/Http/Controllers/ProjectController.php` → metodo `update()`

Completa il metodo `update()`:

1. Aggiungi le stesse regole di validazione del metodo `store()` (TASK 4)
2. Aggiorna il progetto `$project` con i dati validati usando il metodo `update()`

Suggerimento: dopo la validazione, chiama `$project->update($input)` all'interno del blocco `try`.

TASK 7 — Eliminazione in `destroy()`

File: `app/Http/Controllers/ProjectController.php` → metodo `destroy()`

Completa il metodo `destroy()` eliminando il progetto `$project` dal database.

Suggerimento: usa il metodo `delete()` sull'istanza del modello.

Sezione 3 — Dashboard e Grafici

File da modificare: `app/Http/Controllers/HomeController.php`

La home page visualizza sei grafici alimentati da metodi privati del `HomeController`.

I **primi tre grafici** (`chartDepEmp`, `chartGrowth`, `chartGender`) sono già implementati e funzionanti: puoi leggerli come esempi di riferimento. Devi implementare i metodi che producono i **restanti tre**.

TASK 8 — Progetti per dipartimento (`projectsDepartment`)

File: `app/Http/Controllers/HomeController.php` → metodo `projectsDepartment()`

Restituisci una collection dove ogni elemento è un array con:

- `department` → il nome del dipartimento
- `count` → il numero di progetti associati

I dipartimenti **senza progetti** devono comparire ugualmente con `count = 0`.

💡 Guarda come è implementato `departmentEmployee()` — la struttura è identica, cambia solo la relazione da contare.

Suggerimento: `Department::withCount('projects')->get()->map(fn($d) => [...])`

TASK 9 — Distribuzione progetti per numero di dipendenti (`projectsEmployees`)

File: `app/Http/Controllers/HomeController.php` → metodo `projectsEmployees()`

Partendo da `Project::withCount('employees')->get()`, raggruppa i progetti per numero di dipendenti assegnati e costruisci una collection ordinata in modo crescente.

Struttura attesa di ogni elemento:

```
[
  'label' => '3 Employee',    // numero di dipendenti + la stringa " Employee"
  'value' => 7,               // quanti progetti hanno quel numero di dipendenti
]
```

Suggerimento:

```
->groupBy('employees_count')->map(fn($group, $count) => [...])->sortKeys()->values()
```

TASK 10 — Top dipendente per ore: query (topEmployeeProjectHours)

File: app/Http/Controllers/HomeController.php → metodo topEmployeeProjectHours()

Trova il dipendente con il maggior numero di ore totali registrate nell'ultimo anno.

La variabile `$YearAgo` è già definita.

Usa `Employee::withSum()` per sommare le ore dalla tabella pivot `employee_project`, filtrandole per `created_at` compreso tra `$YearAgo` e `now()`.

Ordina per `total_hours` decrescente e prendi il primo risultato con `->first()`.

Alias da usare: 'projects as total_hours'

Colonna da sommare: 'employee_project.hours'

Filtro nella closure:

```
$q->whereBetween('employee_project.created_at', [$YearAgo, Carbon::now()])
```

TASK 11 — Top dipendente per ore: asse temporale (topEmployeeProjectHours)

File: app/Http/Controllers/HomeController.php → metodo topEmployeeProjectHours()

Le variabili `$firstMonth`, `$allMonths` e `$cursor` sono già definite.

Completa il ciclo che popola `$allMonths` con tutti i mesi dall'anno scorso ad oggi nel formato 'm-Y'.

La collection deve contenere **13 elementi** (il mese corrente + i 12 mesi precedenti).

```
while ($cursor->lte(Carbon::now())) {  
    $allMonths->push($cursor->format('m-Y'));  
    $cursor->addMonth();  
}
```

Sezione 4 — Risposta AJAX

File da modificare: `app/Http/Controllers/EmployeeProjectController.php`

Il controller gestisce la creazione della relazione tra un dipendente e un progetto.
Quando la richiesta è AJAX, deve rispondere con JSON invece di eseguire un redirect.

TASK 12 — Validazione in `store()` (`EmployeeProjectController`)

File: `app/Http/Controllers/EmployeeProjectController.php` → metodo `store()`

Completa l'array passato a `$request->validate([...])` con le seguenti regole:

Campo	Regole
<code>employee_id</code>	obbligatorio, deve esistere nella tabella <code>employees</code>
<code>project_id</code>	obbligatorio, deve esistere nella tabella <code>projects</code>
<code>hours</code>	facoltativo, intero, minimo 0

TASK 13 — Risposta JSON in `store()` (`ProjectController`)

File: `app/Http/Controllers/ProjectController.php` → metodo `store()`

Inserisci un blocco `if ($request->ajax()) { ... }`, restituisci una risposta JSON con status `200` avente questa struttura:

```
{
    "success": true,
    "message": "Project creato con successo!",
    "data": {
        "id" : "<id del progetto>",
        "site_name" : "<nome della location>",
        "department" : "<nome del dipartimento associato>",
        "created_at": "<data corrente in formato d/m/Y>",
        "updated_at": "<data corrente in formato d/m/Y>"
    }
}
```

Suggerimento: usa `return response()->json(..., 200)`.

TASK 14 — Risposta JSON in `store()` (EmployeeProjectController)

File: `app/Http/Controllers/EmployeeProjectController.php` → metodo `store()`

All'interno del blocco `if ($request->ajax()) { ... }`, restituisci una risposta JSON con status `200` avente questa struttura:

```
{
    "success": true,
    "message": "WorksOn creato con successo!",
    "data": {
        "employee_name": "<nome completo del dipendente>",
        "project_name": "<nome del progetto>",
        "hours": "<ore inserite>",
        "created_at": "<data corrente in formato d/m/Y>",
        "updated_at": "<data corrente in formato d/m/Y>"
    }
}
```

Suggerimento: usa `return response()->json(..., 200)`.

Per il nome completo usa

```
trim($employee->first_name . ' ' . $employee->middle_name . ' ' . $employee->last_name)
.
```


Consegna

Al termine dell'esercitazione, assicurati che l'applicazione funzioni correttamente in tutte le sue sezioni

Condividi il repository Bitbucket con l'utente del docente e compila il form di consegna con nome, email, matricola e URL del repository.